Força Bruta

Aula 06 - Análise e Técnicas de Algoritmos

Professor Daniel Marques

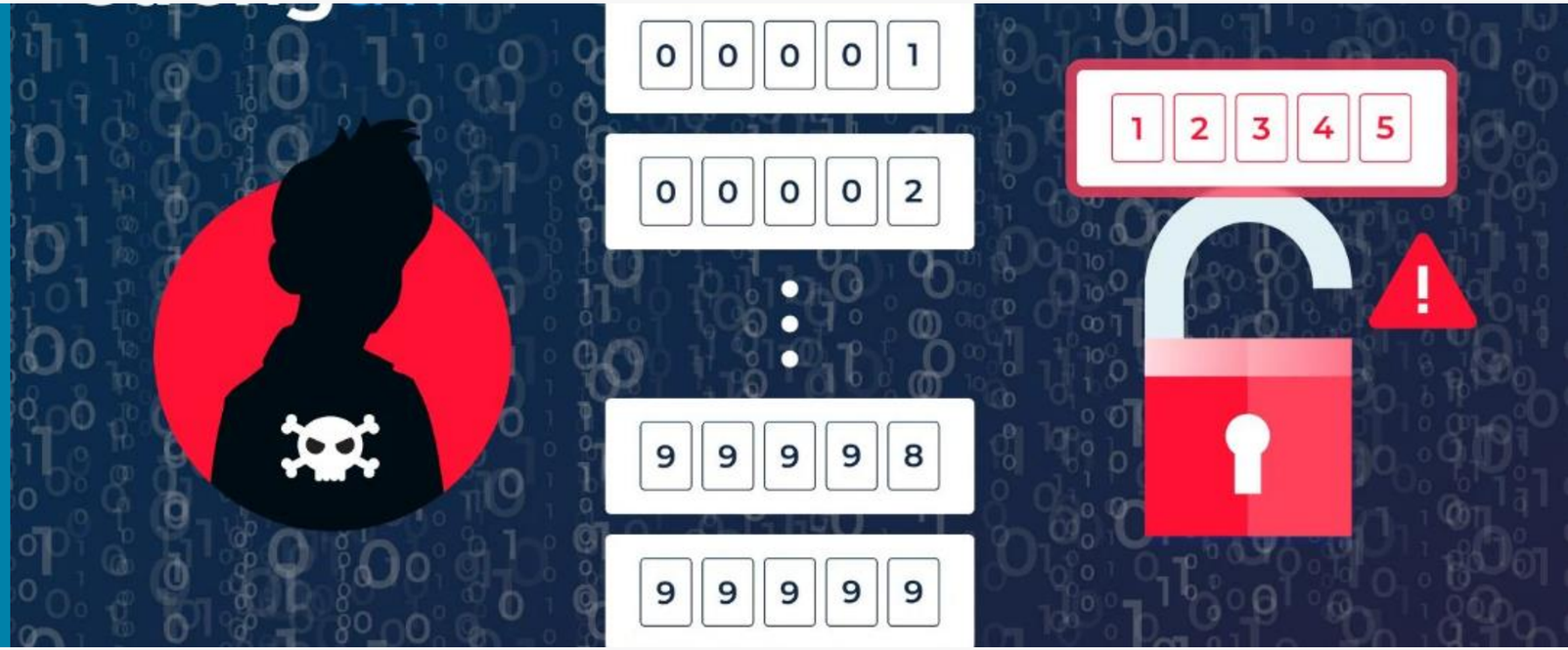
UEPB
Universidade
Estadual da Paraíba

1

Introdução

Introdução

Força bruta (ou busca exaustiva) é um tipo de algoritmo de uso geral que consiste em enumerar todos os possíveis candidatos de uma solução e verificar se cada um satisfaz o problema



Esse tipo de algoritmo geralmente possui uma implementação simples e sempre encontrará uma solução se ela existir. Entretanto, seu custo computacional é proporcional ao número de candidatos a solução que, em problemas reais, tende a crescer exponencialmente

Introdução

A força bruta é tipicamente usada em problemas cujo tamanho é limitado ou quando não se conhece um algoritmo mais eficiente.

eXtreme Go Horse (XGH)

- Força é do computador e não do intelecto
- Normalmente se refere à abordagem mais simples

Introdução

- Raramente resulta em algoritmos inteligentes e eficientes, mas é razoável para

- Normalmente podem ser otimizados com pouco esforço através de heurísticas.

1. Entradas pequenas
2. propósitos teóricos e educacionais
3. Ordenação
4. Busca
5. String matching

2

Ordenação

Ordenação

Considere o problema de ordenar uma lista de elementos em ordem não-decrescente.

Qual o algoritmo mais simples que podemos pensar?

| 89	45	68	90	29	34	**17**
17 |	45	68	90	**29**	34	89
17	29 |	68	90	45	**34**	89
17	29	34 |	90	**45**	68	89
17	29	34	45 |	90	**68**	89
17	29	34	45	68 |	90	**89**
17	29	34	45	68	89	90

1. Ache o valor mínimo na lista
2. Troque-o como o valor na primeira posição
3. Repita os passos acima para o restante da lista (começando na segunda posição e avançando a cada iteração)

Ordenação

Selection Sort

- A operação básica do algoritmo é a comparação (linha 4).
- Achar o menor elemento requer n-1 comparações

```
SELECTIONSORT(A, n)
1  for i = 1 to n − 1
2      min = i
3      for j = i + 1 to n
4          if A[j] < A[min]
5              min = j
6      troca A[i] e A[min]
```

- Achar o próximo menor elemento requer n-2 comparações e assim por adiante.
- Portanto o custo do selection sort é dado por:

$$(n-1)+(n-2)+\cdots+2+1=\frac{n(n-1)}{2}=O(n^2)$$

Ordenação

BubbleSort

Outro algoritmo de ordenação é o BubbleSort

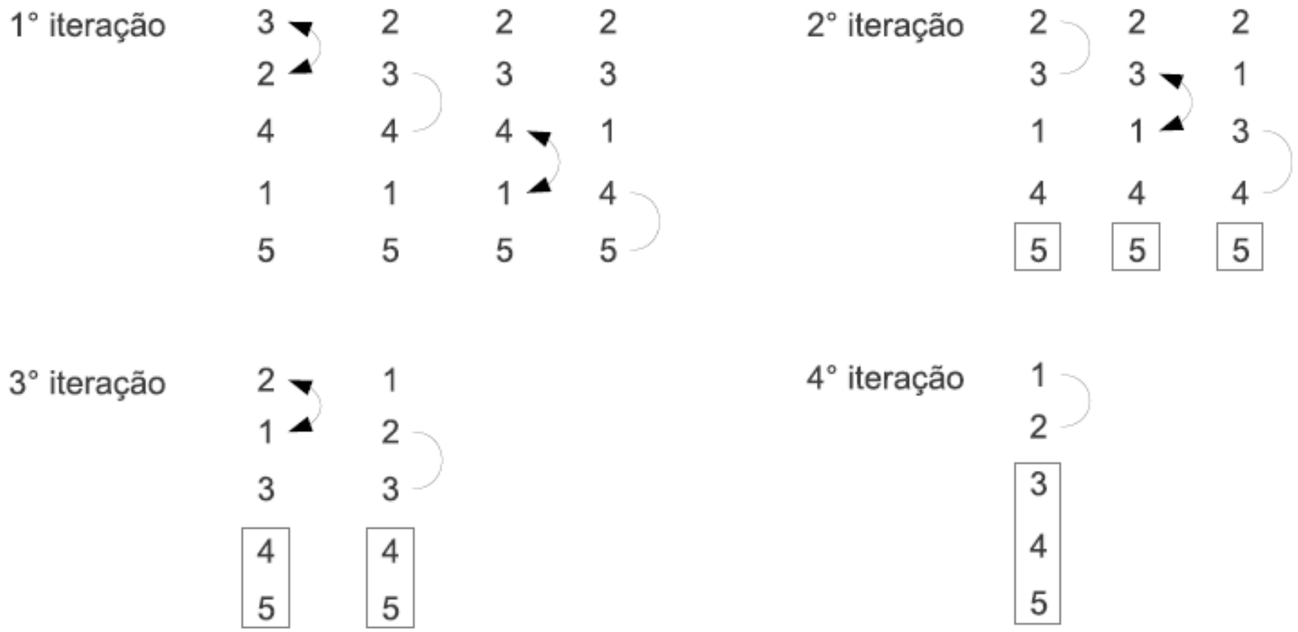
Compara elementos adjacentes sucessivamente trocando-os de posição caso estejam fora de ordem.

```
BOLHA(A, n)
for i = 1 to n − 1
    for j = 1 to n − i
        if A[j] > A[j + 1]
            troca A[j] e A[j + 1]
```

Ordenação

BubbleSort

- **Pior caso = melhor caso = $O(n^2)$**
- **Há como melhorar?**



- **Se numa passada (iteração) nenhum elemento for trocado, parar o algoritmo.**
- **Embora mais rápido para algumas entradas, o custo ainda é $O(n^2)$ no pior e no médio caso**

3

Busca Linear

Busca Linear

- Considere o problema de buscar um elemento numa coleção
- Qual o algoritmo mais simples que se pode pensar para resolver esse problema?

```
BuscaLinear(k, A, n)
i = 1
while (i ≤ n and k ≠ A[i])
        i = i + 1
if i ≤ n
        location = i
else location = 0
return location
```

Compara elementos sucessivos de uma lista com uma chave de busca até que a chave seja encontrada ou a lista é exaurida sem encontrar a chave.

Busca Linear

- $2i + 1$ comparações quando o elemento k está no array
- $2n + 2$ comparações quando k não está

```
BuscaLinear(k, A, n)
A[n] = k
i = 1
while (k ≠ A[i])
        i = i + 1
if i < n
        location = i
else location = 0
return location
```

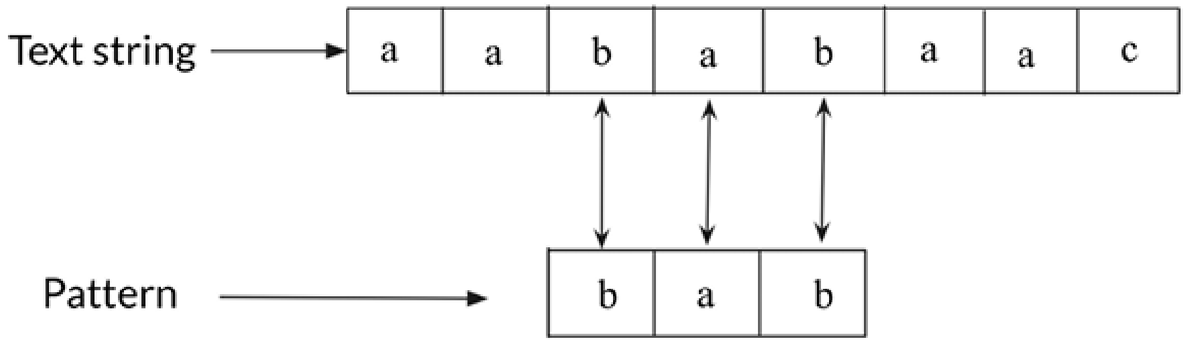
- Podemos fazer melhor?
- Inclua a chave no final do lista!

4

String Matching

String Matching

Achar todas as ocorrências de um padrão em um texto é um problema recorrente em busca textual



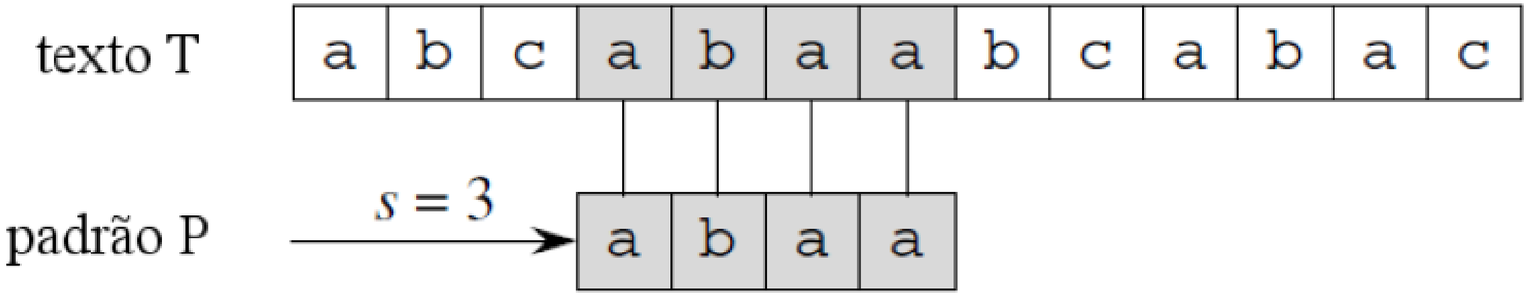
- Algoritmos de string matching também são usados, por exemplo, na busca por padrões em sequências de DNA.
- Busca por determinadas palavras dentro de textos ou códigos (frequência)

UEPB
Universidade
Estadual da Paraíba

String Matching

Notação:

Assumimos que um texto é um array $T[1 \dots n]$ e o padrão um array $P[1 \dots m]$ onde $m \leq n$.



Os elementos de ambos arrays pertencem a algum alfabeto finito Σ e portanto T e P são chamados de strings

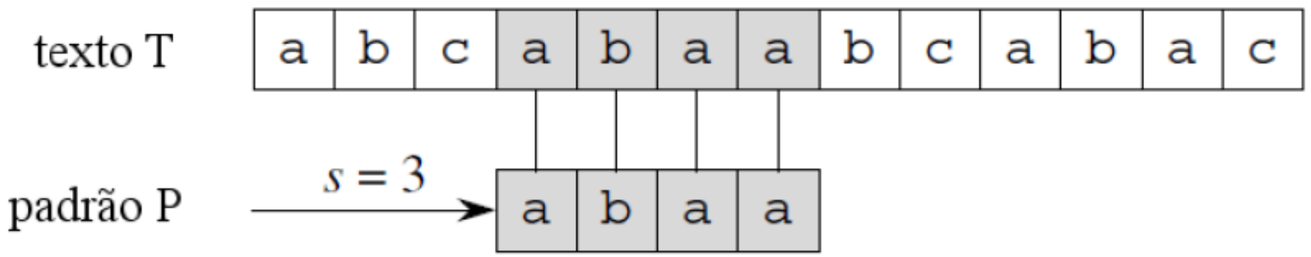
String Matching

O padrão P ocorre com deslocamento s em T se:

$$0 \leq s \leq n - m$$

e

$$T[s+1 \ldots s+m] = P[1 \ldots m]$$



- Se P ocorre com deslocamento s em T, então s é um deslocamento válido.
- Problema: achar todas os deslocamentos válidos com os quais o padrão P ocorre em T.

String Matching

O algoritmo acha todos os deslocamentos válidos usando o laço que verifica se $P[1 \dots m] = T[s+1 \dots s+m]$ para todos os $n+m+1$ valores possíveis de s

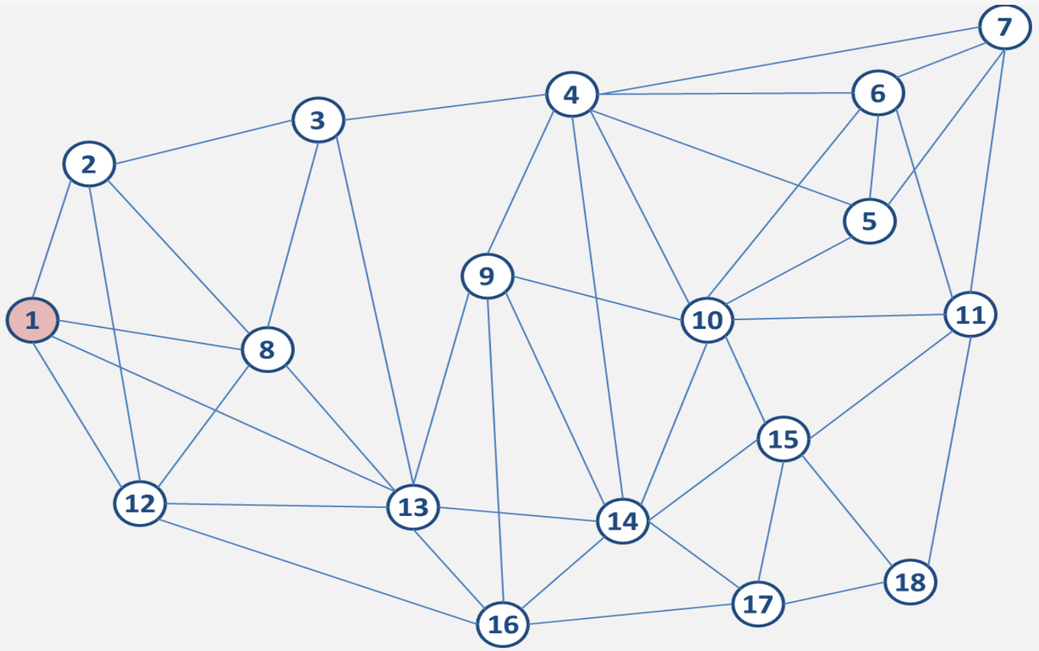
```
STRINGMATCHER(T, P)
1  n = T.length
2  m = P.length
3  for s = 0 to n − m
4      j = 1
5      while j ≤ m and P[j] == T[s + j]
6          j = j + 1
7      if j == m + 1
8          print "padrão ocorre com deslocamento" s
```

- Para cada uma das $n-m+1$ valores possíveis de s, o laço da linha 5 executa m vezes
- Portanto, no pior caso o algoritmo tem custo $O((n-m+1)m)$

Ciclo Hamiltoniano

Problema do caixeiro viajante

Considere um conjunto de n cidades e uma matriz M de tamanho $n \times n$, tal que $M[i,j] = 1$, se existe um caminho direto entre as cidades i e j, e $M[i,j] = 0$, caso contrário.

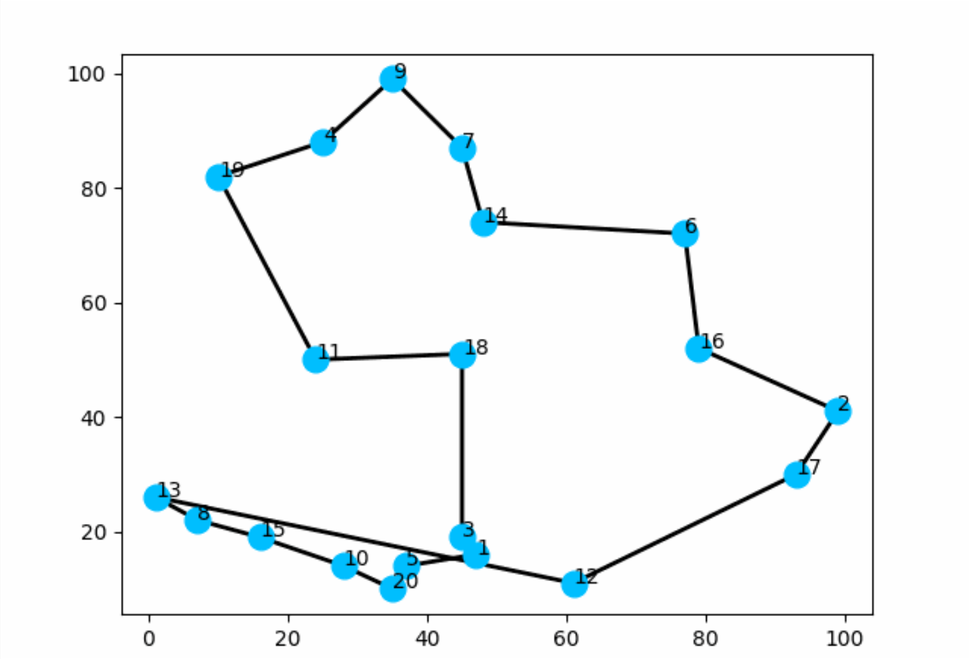


Problema: existe uma forma de, saindo de uma cidade qualquer, visitar todas as demais cidades, sem passar duas vezes por nenhuma cidade, e ao final retornar para a cidade inicial?

Ciclo Hamiltoniano

Problema do caixeiro viajante

Note que, se existe uma forma de sair de uma cidade X qualquer, visitar todas as demais cidades (sem repetir nenhuma) e depois retornar para X,



- Então existe uma forma de fazer o mesmo para qualquer outra cidade do conjunto

Ciclo Hamiltoniano

Como vimos, qualquer cidade pode ser escolhida como cidade inicial.

Sendo assim, vamos escolher, arbitrariamente a cidade n como ponto de partida

x	1	2	3	4	5	6	7	8
1	0	0	1	0	1	1	1	0
2	0	1	0	0	1	0	0	1
3	1	0	1	1	0	1	1	0
4	0	0	1	1	0	0	1	0
5	1	1	0	1	1	0	0	0
6	0	0	1	1	0	0	1	1
7	1	0	0	1	0	1	1	1
8	0	1	1	1	0	1	0	1

- Solução de força bruta: testar todas as permutações das $n-1$ primeiras cidades, verificando se existe um caminho direto entre a cidade n e a primeira da permutação
- assim como um caminho entre todas as cidades consecutivas da permutação e, por fim, um caminho direto entre a ´ultima cidade da permutação e a cidade n

Ciclo Hamiltoniano

1 2 3 4 5 6 7	...	7 6 5 2 3 4 1
1 2 3 4 5 7 6	3 6 4 5 1 7 2	7 6 5 2 4 1 3
1 2 3 4 6 5 7	3 6 4 5 2 1 7	7 6 5 2 4 3 1
1 2 3 4 6 7 5	3 6 4 5 2 7 1	7 6 5 3 1 2 4
1 2 3 4 7 5 6	3 6 4 5 7 1 2	7 6 5 3 1 4 2
1 2 3 4 7 6 5	3 6 4 5 7 2 1	7 6 5 3 2 1 4
1 2 3 5 4 6 7	3 6 4 7 1 2 5	7 6 5 3 2 4 1
1 2 3 5 4 7 6	**3 6 4 7 1 5 2**	7 6 5 3 4 1 2
1 2 3 5 6 4 7	3 6 4 7 2 1 5	7 6 5 3 4 2 1
1 2 3 5 6 7 4	3 6 4 7 2 5 1	7 6 5 4 1 2 3
1 2 3 5 7 4 6	3 6 4 7 5 1 2	7 6 5 4 1 3 2
1 2 3 5 7 6 4	3 6 4 7 5 2 1	7 6 5 4 2 1 3
1 2 3 6 4 5 7	3 6 5 1 2 4 7	7 6 5 4 2 3 1
1 2 3 6 4 7 5	3 6 5 1 2 7 4	7 6 5 4 3 1 2
1 2 3 6 5 4 7	...	7 6 5 4 3 2 1

- Existe uma forma de, a partir da cidade 8, visitar todas as demais cidades, sem repetir nenhuma, e ao final retornar para a cidade 8?
- Existem 5040 permutações das 7 primeiras cidades da lista original

Exercício

Implemente um programa que resolva o problema do Ciclo Hamiltoniano, usando força bruta, onde toda cidade possui

- Uma coordenada cartesiana (x, y). Por exemplo: cidade A (10, 32).
- Utilize uma matriz n x n (onde n é quantidade de cidades), para estabelecer as cidades que possuem ligações com outras cidades
- E encontre o menor caminho do ciclo hamiltoniano

Dúvidas, sugestões ou reclamações?

• • •

Alguma pergunta?

UEPB
Universidade
Estadual da Paraíba

Referências

1. MARINHO, Leandro Balby. Força Bruta. Universidade Federal de Campina Grande. 2018.
2. DIAS, Zanoni. Força Bruta. Universidade de Campinas. 2013.
3.
4.
5.